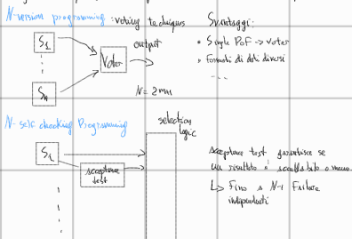


Reliability

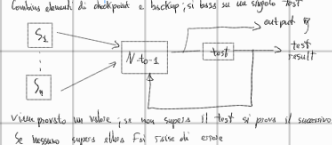
$R(t)$: probability continuous
 $R(t) = P\{s \text{ works in } (t_0, t) | s \text{ worked at } t_0\}$
Unreliability $G(t) = 1 - R(t)$
PDF $f(t) = \frac{dR(t)}{dt} = -\frac{dG(t)}{dt}$
 $\hookrightarrow$ failure rate function $\hat{=}$ number of failures (df) conditions at t $\frac{dR(t)}{dt} \rightarrow$ bathtub function
Possiamo ipotizzare $\lambda(t)$ come costante per l'hardware $\lambda = 1$
 $\hookrightarrow \lambda = -\frac{dR(t)}{dt} \frac{1}{R(t)} \rightarrow R(t) = e^{-\lambda t} \rightarrow f(t) = \lambda e^{-\lambda t}$
 $\hookrightarrow$ MTTF = $\int_0^{\infty} t f(t) dt = \int_0^{\infty} t \lambda e^{-\lambda t} dt = \frac{1}{\lambda}$
 $R_{\text{mem}} = \sum_{i=0}^{\infty} \binom{N}{i} (e^{-\lambda t})^i (1 - e^{-\lambda t})^{N-i} = e^{-\lambda t} + \frac{N}{2} e^{-\lambda t} (1 - e^{-\lambda t}) = 3e^{-\lambda t} - 2e^{-2\lambda t}$
MTTF = $\int_0^{\infty} 3e^{-\lambda t} dt - \int_0^{\infty} 2e^{-2\lambda t} dt = \frac{3}{\lambda} - \frac{2}{2\lambda} = \frac{3}{\lambda} - \frac{1}{\lambda} = \frac{2}{\lambda} < \frac{1}{\lambda} \rightarrow$ more worse than single system but more reliable in first few hours

Software redundancy

Replicare il software è più difficile: stesso software = stessi difetti di implementazione
Design diversity: delle stesse specifiche si realizza con diversi paradigmi, computeri, linguaggi



Recovery block techniques



Distributed testing

Implicit, Explicit
Implementazione di una politica Multilevel security
 $\hookrightarrow$ definizione di secure information flow
how non state information se leggi

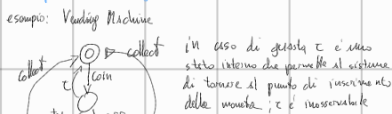
CCS

Calculus of Communicating Processes. Sistemi concorrenti
- set di azioni elementari e operazioni per formare operazioni complesse
- sono processi la τ (operazioni interne e non osservabili)
I processi possono comunicare tramite operazioni complementari;
se no avviene $\rightarrow$ sincronizzazione e formano sistema τ

Representing components of process concurrent

Definire le interazioni
Verificare proprietà

È un tipo di Process Algebra



Axioms
• atomiche
• complessi: atomiche + operatori
• complementari: a e $\bar{a}$. Es: a = invio, $\bar{a}$ = ricezione
 $\hookrightarrow$ sincronizzazione tramite operazione interna τ

Availability

ACS: probabilità che un sistema funzioni all'istante t

Steady-state availability



Consolidation for reliability

Disegnare a blocchi
serie $\rightarrow$ Reliability: $\Pi R_i(t)$
parallel $\rightarrow Q = \Pi Q_i(t) + R = 1 - G(t) = 1 - \sum G_i(t) = 1 - \sum (1 - R_i(t)) \rightarrow \sum_{i=0}^{M+N} \binom{M+N}{i} R^i(t) (1-R(t))^{M+N-i}$
una serie in parallelo: upper bound e lower bound (condizionato su un modulo?)
Upper Bound: si crea il parallelo moltiplicando lo stesso modulo in più paralleli
 $\hookrightarrow R(t) \leq 1 - \Pi (1 - R_{\text{mod}})$
Lower Bound: minimal cut set
 $R_{\text{sys}} \geq \Pi (1 - Q_{\text{cut}}) = \Pi R_{\text{cut}}$

Fault tree

Albero dove la root è l'evento che causa il Failure
Nodi sono altri eventi, combinati con porte logiche AND, OR, NOT
- Minimal cut set: path di eventi che conduce alla root
 $\hookrightarrow$ Si calcola $Q_{\text{root}} = \sum Q_{\text{cut}}(t)$

Timed Automata

Passo sul canale di clock $\rightarrow$ conta il tempo che scappa
Automa: sistemi a stati con transizioni
Transizioni sono composte da:
- guards: espressione da verificare per effettuare la transizione
- azione da svolgere
- uno o più clock reset
Lo stato di un automa è basato sulle locazioni dello stato stesso e dei valori dei clock in un dato istante
Un timed automata modella un sistema in modalità discrete, rappresentate dalle locazioni
Le modalità cambiano come conseguenza delle azioni
Esempio

Definizione Formale di Timed Automata

$A = (L, I_0, C, E, S, I)$

L = location

I_0 = locationi 'iniciali'

C = clocks

E = set di archi tra le locationi

S = set di azioni

I = invariants alle locationi (congiunzioni di condizioni)

Regole di Transizione

Sono transizioni simultanee con stesse azioni e stessi clock

$A = (L_i, I_{0i}, C, E, S, I_i), i=1, \dots, n$

General Semantics

Semantica che descrive in modo formale i sistemi di transizione basati su stati

Nel contesto dei computational tree logic, viene usato per determinare se uno stato rispetta una data formula ϕ

Reachability in a stop: dato uno stato, è possibile raggiungere un altro dato spazio con uno stop

$\hookrightarrow$ è possibile raggiungere con i stop uno stato che soddisfa ϕ

Unreachability: come sopra ma con un numero finito di stop

Unreachability: dati i e_j , partendo da i , allora j sarà presente in ogni path tra i e la fine

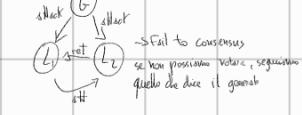
$\hookrightarrow$ in ogni path tra i e la fine, raggiungeremo uno stato che soddisfa ϕ

Byzantine Problem $\rightarrow$ consensus

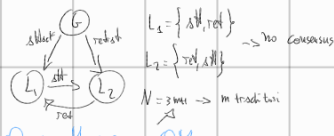
Voting function: I_1 : tutti i voti obbediscono allo stesso comando

I_2 : i voti devono essere d'accordo con il generale, se reale

Cse: i host



Cse: general



Orsi Message OM

$\hookrightarrow$ trasmissione del messaggio da tutti verso tutti

$\hookrightarrow$ Voting system

Infeasible: $O(n^m)$

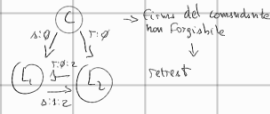
$\hookrightarrow$ Aggiungiamo firme del generale

Signed Message

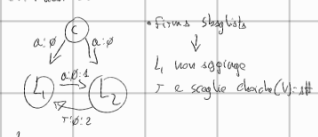
$\hookrightarrow$ clorche (V) $\rightarrow$ ret se $V \neq v$ Diet se $V \in \{v_1, v_2\}$

- 1) $V_i = \{v\}$
- 2) C firma e manda il suo valore a tutti
- 3) Per ogni i , aggiungo o meno il valore v a V_i in base alla sua firma
- 4) choice

ES: i general traitor



ES: i host traitor



$\hookrightarrow N=2M+1$

Informative Redundancy

Possiamo aggiungere o incorporare informazioni

Aggregare informazioni

Detection

- parity bit: n bit + 1

$\hookrightarrow$ code words: error

- complement: n bit

$\hookrightarrow$ aggiungo i bit complementati

- Hamming Distance

$\hookrightarrow$ numero di bit per i quali due code word differiscono

se K :

- detection di $K-1$ single errors

- correction di d : $K=2d+1$

- check summing: somma di tutti i bit

- codici strutturati

Correction

- 2-dim parity-check

- Hamming distance

- parity bit: potenza di 2

$\hookrightarrow$ ognuno dei bit è composto da n bit la cui posizione ha la cifra significativa pari a 1

es: per bit 1: 1, 3, 5, 7, ...

per bit 2: 2, 3, ...

- Self-checking circuitry

- self testing: ogni fault single è detectabile

- fault scan: output è sempre una code word o una non-code

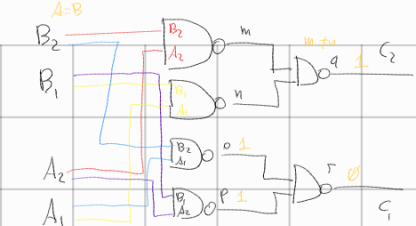
$\hookrightarrow$ total self-checking: se implemento entrambi i precedenti

Two input TSC computer

Circuito a due input; output è se uguale, 1, altrimenti

$\hookrightarrow$ lo costruiamo con le NAND: A_1, A_2, B_1, B_2

01	1
10	2
11	3
00	4
01	5
10	6
11	7



NAND

A	B	C
0	0	1
0	1	1
1	0	1
1	1	0

se per errore $A_1 \neq A_2$ o $B_1 \neq B_2 \rightarrow$ fault detection

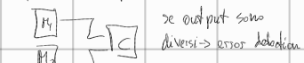
Hardware Redundancy

Passive

- TMR
- NMR

Active

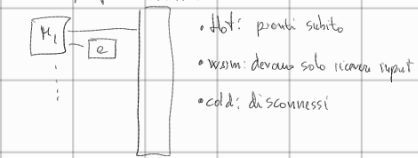
= Duplication with Comparator



= Reconfigurable Duplication



= Stand-by spares



= Pair And spares



Approcci ibridi

= NMR riconfigurabile

